

Genome Analysis labs

Student Manual

[Introduction](#)

[Project Planning](#)

[Project Organization](#)

[Keeping the metadata](#)

[Getting familiar with your data](#)

[Organising your working directory](#)

[Working with computers](#)

[Working on the local computers](#)

[Working on UPPMAX](#)

[Connecting to a working node](#)

[Submitting batch jobs](#)

[Using UPPMAX modules](#)

[Transferring files between UPPMAX and the local computers](#)

[Where should I run my analyses?](#)

[Working with GitHub](#)

[What should you include in your GitHub repository?](#)

[Getting started with GitHub](#)

[Creating a repository](#)

[Local vs remote repository](#)

[Building up your repository](#)

[Updating your repository](#)

[Papers - short summary](#)

[Paper I](#)

[Paper II](#)

[Evaluation](#)

[Paper I - Zhang et al. 2017](#)

[Paper II - Zhou et al. 2023](#)

[Practical Information About Software](#)

[Reads quality control](#)

[Reads preprocessing](#)

[Genome and Metagenome Assembly](#)

[Assembly evaluation](#)

[Binning](#)

[Binning evaluation](#)

[Annotation](#)

[Homology search](#)

[Phylogenetic placement](#)

[Mapping](#)

[Post-mapping analyses](#)

[Read counting](#)

[Expression analyses](#)

[Visualisation of the genomes](#)

[Genome scaffolding](#)

[Appendix I: Software list](#)

[Appendix II: Estimated running time for analyses](#)

[Appendix III: Analyses checkpoints](#)

Introduction

The technological advancement around the usage of sequence data has surpassed all expectations in the last decades. The development of new sequencing strategies, hand in hand with the continuous release of improved bioinformatics software, is promising and unstoppable. The amount and breadth of both DNA and RNA data accrue in public databases. However, obtaining high-quality biological information from sequences is not straightforward, and, without proper care, the fast growth of sequence data can cause quantity to obliterate quality. For this reason, a deep understanding of bioinformatics methods, coupled with an educated interpretation of their results, is necessary to unleash the biological information concentrated within sequence data. This strategy opens doors to an incredible number of applications, including biomedical advances, environmental conservation or bioengineering, and uncovers new fundamental insights on biological diversity, evolutionary history, physiology, etc. In the proper hands, sequencing reads become knowledge goldmines.

One of the aims of these labs is to get familiar with bioinformatics tools and methods that are commonly used when analysing sequencing data. We will do so by working with real-life genomic and expression data, and applying state-of-the-art methods. You will start off by selecting a recent scientific article, and will proceed by re-analyzing their data in a similar way as the authors did on the original study, and re-evaluating their biological conclusions.

The first task at hand will be to plan the analyses that you need to perform. It is very important that you understand the data that you will be working with, and to have clear aims to pursue. You will **adapt the analyses to the available tools and your own interests**. Therefore, those of you starting from the same data can follow different paths and produce different results. Think creatively, and work carefully: feel free to design your own workflow and produce your own scripts towards your particular interests, and introduce the right check-ups to ensure that each step works as expected. Moreover, note that papers often lack important details needed to reproduce their analyses, and some software may require specific concepts or parameters that are not straight-forward at a first glance. You will need to face these problems and try to find your way around it. In those cases remember that Google is your friend :-).

As mentioned above, you are going to use the real data from the articles you chose. All the data can be found in the Sequence Read Archive (<https://www.ncbi.nlm.nih.gov/sra/>). See the 'Getting familiar with your data' section for more information. The tasks you will perform will often be computationally demanding, so you will need to run them in computer clusters. Therefore, you will use UPPMAX for most of the analyses. See the 'Working on UPPMAX' section for more information. Finally, an important task during any scientific project is to document all the analyses you have done. To do so, you will create a GitHub repository where you will record codes and used methods. See the 'Working on GitHub' section for more information.

Project Planning

Good project planning and data organisation are integral parts of any research project. A research plan can help you understand your project better and make you aware of issues and bottlenecks that can occur in the process. This will help you avoid getting lost in the data and be more efficient. Creating a project plan will be the first thing you will do in these labs. Your project plan should address at least the following points:

- What is the aim of your project? What question(s) do you want to answer with your research?
- What type of analyses will you perform in order to answer these questions? And in which order? Which softwares will you use? Are there any time bottlenecks? If so, can you identify any analyses that will require longer times?
- What is the time frame for your project? Can you define some time checkpoints for when you should have finished certain analyses? When do you need to have finished running all the softwares so you can start to analyse the data?
- What types of data will you be handling? How much space do you need in order to store the data? (You might not know that in advance, but pay attention to this as you work and manage your available space!)
- How will you organise your data? (see “Project Organization”)

To make your project plan you will need to have **read and understood the paper** that you have chosen before the first lab session. It is extremely important to frame the questions you want to answer at early stages. Since you are going to re-analyze data, there will be certain things you will depend on and that cannot be changed, such as the data available or the experimental design. Make sure that you know what data you can use and **identify important information** such as the type of data (RNA-seq, Whole Genome Sequencing, short-reads/long-reads, etc), the biological source (tissue, sample, etc), and how it was generated (type of libraries, special adapters, potential issues, etc). These details will determine the choice of certain analyses or software over others. Once you have identified your aim and are familiar with your data, you can start planning the approach you will follow. A very intuitive way of doing this is by drawing a diagram in which the input/output data for each step is connected by the analyses to perform and the tools to use, such as the one in **Figure 1**.

At this point it is also important to try to spot **steps that can take a long time**, so you can optimise your work and avoid long waiting times. You may also need to account for **unexpected results or errors** that will require your strategy to be revised, so make sure to introduce a certain degree of flexibility in your project plan. You will work on this plan during the first lab session and the TAs will help you assess and polish it.

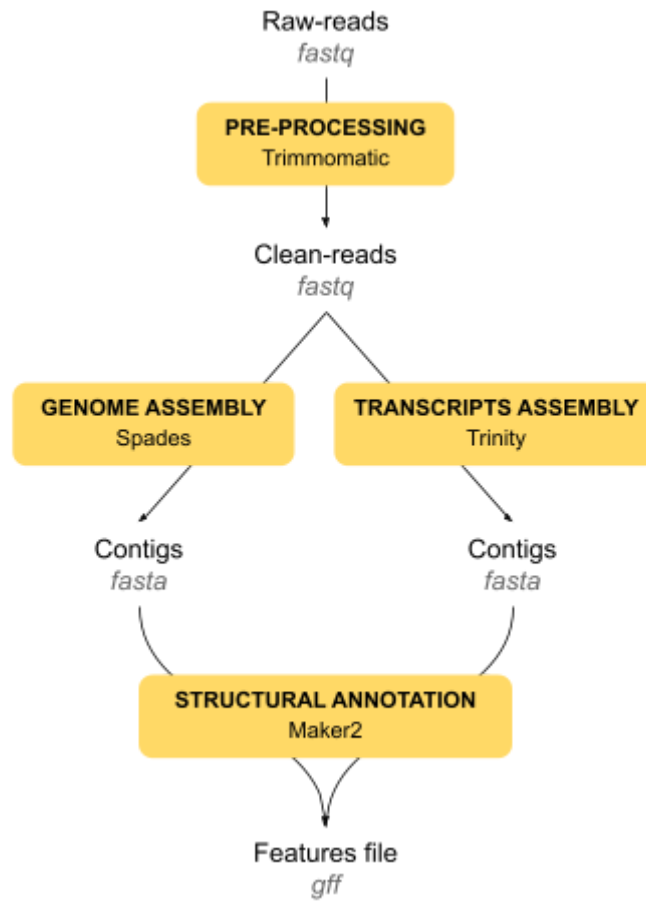


Figure 1. Diagram representing input/output workflow

Project Organization

Since you will work with large amounts of data it is extremely important to **keep a good data structure from the very beginning**. A good project organisation will make your work easier as you generate more and more data and will help your “future you” and others understand your project. There are many things that we could discuss here, but we will focus on the importance of keeping clear metadata, and include a few tips about how to organise your working directory.

Keeping the metadata

Metadata refers to information about data, such as how or where it was collected. In your case this will be the information associated with the sequencing data and how it was generated. The process of collecting metadata starts in the lab, before the sequence data are produced. This information is as important as the sequences themselves, since without it the data are useless.

Since you will work with public data, you can obtain the metadata associated with the sequencing reads files from public databases and from the original article. There are many ways to store the metadata, but an easy one is to use a spreadsheet (like Excel, LibreOffice or Google Sheets). Since these **tables will be read by computers you need to keep them structured to avoid parsing errors**. The data to include will depend on each experiment. These are few guidelines to keep in mind when creating a spreadsheet table with the metadata for your sequence files:

- Write one sample per row and one variable per column. For example, all information about the SRR00001 should be on the same row where columns are variables such as: SRA_identifier, tissue, stage, type_of_sequencing, etc.
- Do not merge cells!
- Avoid spaces! Use '-' or '_' instead.
- Be consistent
- Use informative names
- Save the data in plain text files as .tsv (tab-separated values) or .csv (comma-separated values)

Getting familiar with your data

The data you will be working with comes from the NCBI Sequence Read Archive (SRA), a repository of next-generation sequencing data. In order to avoid exceeding our space allocations, we have downloaded, trimmed and, in some cases, subsampled these read files for you. These files are named according to their SRA accession, which can also be found on the paper. To learn more about your data you can follow the following steps:

1. Go to the <http://www.ncbi.nlm.nih.gov/sra> page and search for the SRA accession of your samples. If you click on the entry of interest you can already start gathering some information. Take some time to examine the page.
2. Under “Study”, click on “All Runs”. This page will show all datasets related to the experiment. Inspect the information given on this page.
3. In the “Select” table, under “Download” click on “Metadata”. This will download a file called SraRunTable.txt that contains information about the whole dataset (not the actual data).

Organising your working directory

There is not one unique way of organising a working directory that works for every person and every project. However, there are few guidelines that can be good to keep in mind:

- **Data and code should be separated.** Ideally you would like to keep your code in a repository with version control. Small data files, such as final results, figures and text

can also be included in your repository, but never big data files (there are other repositories that you could use if you wanted to make that data publicly available).

- Keep data files with unique and **informative names**. The read files with which you will be working are named as their SRA accessions. It is useful to give these files unique and meaningful names so we can recognise what they are right away. However, it is advisable to always keep the raw data with the original names. In those cases, we can use symbolic links that create shortcuts to the original files but that have different names. That can be done with a command such as:

```
$ ln -s /path/to/original/file /path/to/new/file
```

- It is handy to generate folders or file names that start with a number, since by default they will be shown in alphanumerical order. It is easier to know in which order they were created when they are numerically organised.
- Data files, especially big data files, should be **compressed**. For example, files in FASTQ (sequences with quality scores) or SAM (mapped sequences) format are usually huge, and we normally have a limited amount of storage resources. For example, your home directory on UPPMAX (where you will be saving your analyses) can store up to 32 Gb. Having compressed files is not a problem for most tools and, for those tools that cannot handle it, we can uncompress files on the fly and use the result as standard input to the software. Here are some examples of how to do that:

```
# Uncompressing and piping
$ zcat sampleA.fastq.gz | wc -l

# Uncompressing and input as a file
$ SoftwareX <(zcat sampleA.fastq.gz)
```

An example of an appropriate working directory structure could be the following, using the command “tree” (note: ‘.gz’ files are compressed; file names followed by “->” are symbolic links):

```

genome_analyses/
├── analyses
│   ├── 01_preprocessing
│   │   ├── trimming_software
│   │   │   ├── conditionA.trim.fastq.gz -> ../../../../data/trimmed_data/SRR000001.trimmed.fastq.gz
│   │   │   └── conditionB.trim.fastq.gz -> ../../../../data/trimmed_data/SRR000002.trimmed.fastq.gz
│   │   ├── fastqc_raw
│   │   │   └── fastqc_report.txt
│   │   └── fastqc_trim
│   │       └── fastqc_report.txt
│   ├── 02_genome_assembly
│   │   ├── assembly_softwareA_settingsX
│   │   └── assembly_softwareB_settingsX
│   └── 03_structural_annotation
│       └── annotation_software_settingsX
├── code
│   ├── 0_gather_data.sh
│   ├── 1_preprocessing.sh
│   ├── 2_genome_assembly.sh
│   └── 3_structural_annotation.sh
├── data
│   ├── metadata
│   │   └── sample_information.csv
│   ├── raw_data
│   │   ├── SRR000002.fastq.gz
│   │   └── SRR000001.fastq.gz
│   └── trimmed_data
│       ├── SRR000001.trimmed.fastq.gz
│       └── SRR000002.trimmed.fastq.gz

```


Working with computers

During these computer labs you will work either on the local computers or on the UPPMAX cluster. In both cases, you will work on a Linux environment.

If you have any doubts about how to do that, please check [this introduction to Linux](#) and [these linux exercises](#). And, as we said before: Google is your friend! You can find answers online about how to do almost anything in Linux.

Moreover, if you want information about specific programs, you can obtain much of that from the program itself. For **every new command you run**, try searching for manuals (program 'man') or help options (options '-h' or '-help'), e.g.:

```
$ man ssh
$ blastp -help
```

Note: In UPPMAX you need to allocate a node before you run these commands (see "Working on UPPMAX").

To start the local computer in Linux mode:

Switch on the computer (or reboot if it has been started in a Windows session). When the blue 'hp' screen appears at the beginning of the process, press F12 and the computer will switch to Linux.

Working on the local computers

Most of the analyses should be run on UPPMAX, since it offers much higher computational power, memory and storage space.

However, some softwares are better run on local computers. This is especially true for graphics intensive programs such as Integrated Genome Viewer (IGV), Artemis, Artemis Comparison Tool (ACT) and R studio.

Whenever you want to run a software on a local machine using as input a file which is on UPPMAX, you must first transfer that file to the local machine. You will learn how to do that in the next section.

Sometimes you might need to access the university network from your own computer. If your computer runs Linux or macOS, you can ssh from the terminal with:

```
$ ssh -AX username@solander.ibg.uu.se
```

Depending on your distribution, you may first have to install ssh-client. On Windows you can use MobaXTerm or PuTTY ([see instructions here](#)) to connect to IBG's server

(solander.ibg.uu.se). Be aware that not all software is accessible when working remotely on solander.

Working on UPPMAX

UPPMAX is Uppsala University's computer cluster for large-scale computation and data storage. We will do most of our work and store most of our files on UPPMAX. To do that, we use ssh (Secure SHell), a program that allows us to connect remotely to other computers and gives us access to a command-line on them.

To connect to UPPMAX from linux, open a terminal and type:

```
$ ssh -AX username@pelle.uppmx.uu.se
```

Replace 'username' with your UPPMAX user name.

NOTE: The option '-X' activates X-forwarding on the connection, allowing for graphical data to be transmitted if a program requests it, i.e. programs can use a graphical user interface. If you are using Windows, you might also have to select an option to "enable X11 forwarding" in MobaXTerm, PuTTY or similar software if you want to run graphical interfaces remotely. You can do the same when you log in to any remote server, for example, IBG's server.

To log in to the Pelle cluster used in this course, both your password and two-factor authentication (TOTP) are required. If you haven't set up TOTP yet, you can follow [this](#) guide.

After introducing your password and TOTP you should see something like what is shown in **Figure 2**:

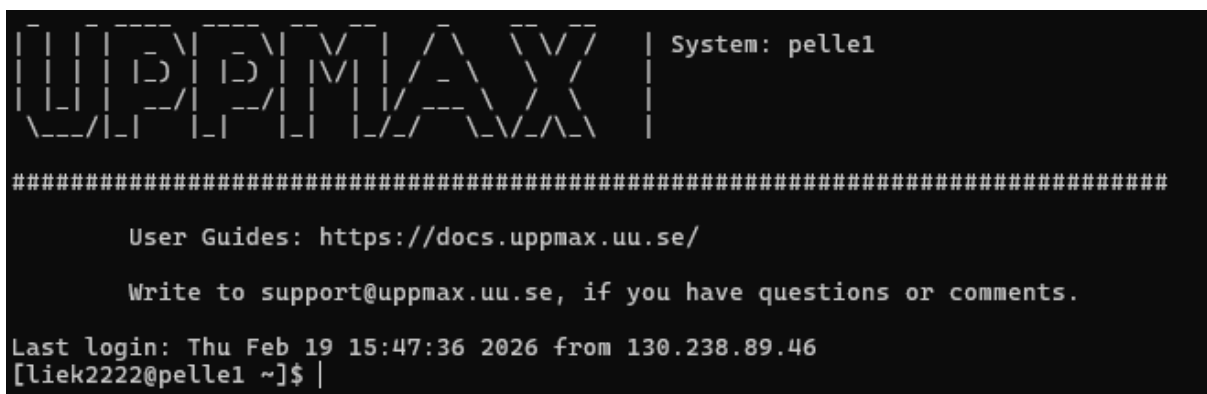
A terminal window showing the UPPMAX login banner. The banner features the word 'UPPMAX' in a large, stylized, dashed font. To the right of 'UPPMAX' is the text 'System: pelle1'. Below this is a line of asterisks. Further down, it says 'User Guides: https://docs.uppmx.uu.se/' and 'Write to support@uppmx.uu.se, if you have questions or comments.' At the bottom, it shows the last login information: 'Last login: Thu Feb 19 15:47:36 2026 from 130.238.89.46' and the prompt '[liek2222@pelle1 ~]\$ |'.

Figure 2. Welcome message from Uppmax. This is your home directory.

This welcome banner means that you have connected to a login node in UPPMAX. When you log in you enter one of the login nodes (a node is basically a PC-unit with its own processor cores, memory, storage etc within a computer cluster like UPPMAX).

Note that **login nodes should NOT be used for computation**, and UPPMAX staff will send angry emails if that happens. In order to run jobs, you have two options:

- Allocate a working node (see 'Connecting to a working node'), which will allow you to run programs directly from the command line.
- Submit batch jobs (see 'Submitting batch jobs')

Importantly, the software packages are made available to the users in UPPMAX as a module system, requiring that you load specific modules in order to run software (see 'Using UPPMAX modules').

Connecting to a working node

During the labs, we will use UPPMAX reservations (see Appendix II) that will allow us to work with priority allocating nodes. Keep in mind that jobs that are submitted using reservation codes must be completed before the end of the lab, or else they will be automatically cancelled. If a process requires longer running times, it needs to be submitted as a batch job.

To ask for a slot in a computation node (we will only allow two cores per person), type:

```
$ salloc -A uppmx2026-1-61 -c 2 -t 00:00:00
--reservation=<code>
```

And include the amount of time asked for under the option '-t', and, the correct reservation code (see Appendix III). Now computation nodes on the pelle cluster will be issued. (More details on using pelle are here https://docs.uppmx.uu.se/cluster_guides/pelle/). Once you are granted access, the commands you type will be done on that node and not the login node. If you finish before the allocated time has expired, type 'exit' to release the slot.

To ask for a job allocation after lab hours remove the --reservation=<code> option from the previous command:

```
$ salloc -A uppmx2026-1-61 -c 2 -t 00:00:00
```

If you want to use the graphical interface of a programme, you can use interactive instead of salloc. For this to work, you have to use the -X option when you ssh into UPPMAX.

NOTE: When the time you requested is over, you will be automatically disconnected from the node and any programs that are running will be interrupted, possibly causing loss of data!

Submitting batch jobs

Sometimes analyses take a long time to run, and the time we have available in the computer labs is limited.

In such cases, sending a batch job can be very handy, as the jobs will run even when you are not connected to UPPMAX (i.e: you can run time consuming analyses overnight). To do that, you will write a script containing submission information and the commands to be run. You will need to create a text file with the following format:

```
#!/bin/bash -l

#SBATCH -A uppmx2026-1-61
#SBATCH -p pelle
#SBATCH -c 2
#SBATCH -t 00:00:00
#SBATCH -J job_name
#SBATCH --mail-type=ALL
#SBATCH --output=%x.%j.out

# Load modules
module load ....

# Your commands
<Command_1...>
<Command_2...>
```

Make sure you change all relevant information on the header, such as:

-c Number of cores (do not use more than 2 unless specified in the software section!). **Important!** By using this flag you will reserve the number of cores, but you still need to specify the number of threads (cores) to use **on the command** of the software to run (e.g. `blastp -num_threads 2`). Otherwise you will still be using a single core, even though you have reserved more.

-t How much time you wish to allocate to this job (dd-hh:mm:ss)

-J A name for your job

--mail-type The system will send notifications to the e-mail address associated with your SUPR account when jobs have started, finished or if they fail. Useful for monitoring your jobs.

NOTE: When the amount of hours you assigned to the job is over, the job will be cancelled.

Once you have written your script you need to submit it to the cluster using the following command:

```
$ sbatch sbatch_script.sh
```

To see the status of your submitted jobs you can run the following (replace “username” with your own username):

```
$ squeue -M pelle -u username
```

Using UPPMAX modules

To run a program on UPPMAX, first we need to upload its module, otherwise the program will not be found by the computer. To do that, type:

```
$ module load <module_name>
```

To see which modules are available in UPPMAX, type:

```
$ module avail
```

If we do not remember the complete name of a module or its spelling, we can use ‘module spider’. ‘module spider’ can be used as a searching engine for other modules. It has the advantage of being case-insensitive.

```
$ module spider <search_term>
```

For example if we want to run BCFtools we can type:

```
$ module spider bcftools
```

Which outputs information about the available versions:

```
[liek2222@pelle1 ~]$ module spider bcftools

-----
BCFtools:
-----
Description:
  BCFtools is a set of utilities that manipulate variant calls in the Variant Call Format (VCF) and its binary counterpart BCF

Versions:
  BCFtools/1.3.1-GCC-13.3.0
  BCFtools/1.13-GCC-13.3.0
  BCFtools/1.17-GCC-13.3.0
  BCFtools/1.19-GCC-13.3.0
  BCFtools/1.20-GCC-13.3.0
  BCFtools/1.21-GCC-13.3.0
  BCFtools/1.22-GCC-13.3.0

-----
For detailed information about a specific "BCFtools" package (including how to load the modules) use the module's full name.
Note that names that have a trailing (E) are extensions provided by other modules.
For example:

  $ module spider BCFtools/1.22-GCC-13.3.0
-----
```

And we can load the latest version using:

```
$ module load BCFtools/1.22-GCC-13.3.0
```

After that you can call BCFtools simply by typing:

```
$ bcftools
```

NOTE: All modules are unloaded when you disconnect from UPPMAX, so you will have to load the modules again every time you log in. If you load a module in a terminal window, it will not affect the modules you have loaded in another terminal window, even if both terminals are connected to UPPMAX. Each terminal is independent of the others.

To view which module you have loaded at the moment, type:

```
$ module list
```

To unload a module, use:

```
$ module unload <module name>
```

And to unload all loaded modules, type:

```
$ module purge
```

Transferring files between UPPMAX and the local computers

There are several programs that you can use to transfer files between UPPMAX and the local computers, such as 'scp' or 'rsync'. The general way to do it (rsync example) is the following.

To transfer a local file to a remote computer:

```
$ rsync -P file user@machine:/destination/path/  
$ scp file user@machine:/destination/path/
```

To transfer a file in a remote computer to our local machine:

```
$ rsync -P user@machine:file /destination/path/  
$ scp user@machine:file /destination/path/
```

If we want to transfer from UPPMAX to our local computer we will do something like:

```
$ rsync -P username@pelle.uppmx.uu.se:/path/to/your/file .  
$ scp username@pelle.uppmx.uu.se:/path/to/your/file .
```

Note that you always have to run this on the terminal on your local computer (not on UPPMAX).

Where should I run my analyses?

All analyses should preferably be run in your home folder at UPPMAX (/home/<your_username>). Inside your home you can create folders and organise your files as you wish. The home folder is limited to 32Gb, and for some analysis, this might be insufficient. In this case, you can also create a personal folder within the project directory (under /proj/uppmx2026-1-61/nobackup/work/). Please note that files in this directory are readable (and writable) to everyone in the course. Also note that if you want to keep anything after the end of the course, you need to save it either to your /home or to your private computer.

The “raw data” you will use in your projects is stored in the project folder on UPPMAX (/proj/uppmx2026-1-61/Genome_Analysis/Name_of_your_paper/DNAorRNA_folder).

These files are very big, so **do not move/copy them** to your home or they will use up a very large part of the total disk space you have. Instead, create soft links to them (be sure to create a link to the INDIVIDUAL files, do not create a softlink to the folder) using “ln -s”, e.g.:

```
$ cd /home/<your_username>
$ ln -s /proj/uppmx2026-1-61/Genome_Analysis/Name_of_your_paper
./raw_data
```

BEFORE YOU PROCEED verify that the file are soft links by:

```
$ ll -a
```

It should return something looking like this:

```
total 24
drwxrwsr-x 2 borlotti borlotti 4096 7 mar 11.24 .
drwxrwsr-x 4 borlotti borlotti 4096 13 apr 2023 ..
lrwxrwxrwx 1 borlotti borlotti 99 7 mar 11.24 SRR4342129_1.paired.trimmed.fastq.gz -> /proj/uppmx2025-3-3/Genome_Analysis/3_Thrash_2017/DNA_trimmed/
SRR4342129_1.paired.trimmed.fastq.gz
lrwxrwxrwx 1 borlotti borlotti 99 7 mar 11.24 SRR4342129_2.paired.trimmed.fastq.gz -> /proj/uppmx2025-3-3/Genome_Analysis/3_Thrash_2017/DNA_trimmed/
SRR4342129_2.paired.trimmed.fastq.gz
lrwxrwxrwx 1 borlotti borlotti 99 7 mar 11.24 SRR4342133_1.paired.trimmed.fastq.gz -> /proj/uppmx2025-3-3/Genome_Analysis/3_Thrash_2017/DNA_trimmed/
SRR4342133_1.paired.trimmed.fastq.gz
lrwxrwxrwx 1 borlotti borlotti 99 7 mar 11.24 SRR4342133_2.paired.trimmed.fastq.gz -> /proj/uppmx2025-3-3/Genome_Analysis/3_Thrash_2017/DNA_trimmed/
SRR4342133_2.paired.trimmed.fastq.gz
```

We recommend that you use the project folder to place only big files such as BAM files, and also that you keep track of the space that is still available to you in your home folder before and after you run analyses. To check the space that is available to you, you can use:

```
$ uquota -u <your_username>
```

Working with GitHub

GitHub (<https://github.com/>) is a code-hosting platform for version control and collaboration. It allows you to keep and manage multiple parallel versions of a document, and provides you with a platform to invite people to collaborate and to make your work available to others, if you want to share it.

In our project, you will use **GitHub to document and keep track of your analyses and code**. Good documentation allows you to troubleshoot problems, repeat analyses when needed, and allows others to see and understand what you have done. It is through the GitHub documentation that the course teachers can see what you are doing, and how they can help you figure out what might be going on whenever you have a problem with the project.

For these reasons, **it is important that you create a GitHub page which is comprehensive, regularly updated and well organised**. This will be part of the evaluation of your project (see the 'Evaluation' section).

What should you include in your GitHub repository?

You can keep any project-related documentation here, but most importantly you are expected to use GitHub to save your code and to keep an updated wiki on your project. You should save all code you use, be it software commands, a simple bash one-liner, a longer Perl/Python script, an UPPMAX batch script, etc.

Moreover, your GitHub wiki is where you can write explanations about what you are doing, why you are doing it, and what you plan to do. It should include:

- Which paper you are working with
- Your project plan
- Your goals/hypotheses
- A section for each analysis you run (quality control, assembly, RNA mapping, etc.) with explanations on methods you are using, the results you got, a short discussion about them, etc.
- Any general thoughts, discussions, speculation, etc. that you may have about the project and your overall results
- Also suggested: A daily log of what you did on each day of work

IMPORTANT NOTE: Both code and wiki must be written in a clear and organised way, with enough explanations for another person to properly understand and reproduce what you are doing. Consider that your documentation should be clearly understandable to:

- Yourself, if you need to check what you did yesterday or 3 weeks ago
- Your classmates, if they want to read and discuss your project

- Your course teachers, so that they have an idea of your progress and can help you with questions and problems you may run into. Moreover, this documentation will serve as a full report of your work and your interpretation of your results, and will be evaluated as such.

Getting started with GitHub

Here we make a summary of how to use some of the basic features of GitHub, but we strongly recommend you to have a look at the links below when starting with GitHub for the first time or when trying to find answers to questions related to the system.

[This site](#) provides answers to a large number of GitHub questions, and can be a good starting point when trying to learn how to do something on the platform. Other tutorials can be found [here](#), and [here](#).

But before going into details about creating and managing your repository, we'll briefly introduce the environment you'll be working on.

As shown in **Figure 3**, GitHub is a multilevel interactive environment. Everything will be stored in your **remote repository**, that will exist in the GitHub cloud. However, you will perform all your analyses from the **working copy** located in your working directory (i.e. your home folder in UPPMAX), and you will eventually transfer the data you want (e.g. your code) to the remote repository. Between you and the cloud there are two extra intermediate levels: the **local repository** (that you have to actively create) and the **index** (also known as the **staging area**, that acts as a cache between you and the local repository). All these levels are connected to each other through different functions. We will explain to you the most important things that you will use for this project, but you can explore on your own all the possibilities that this virtual environment offers.

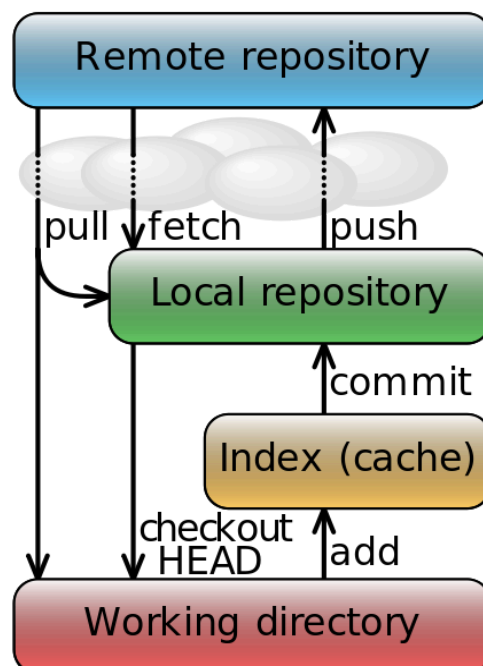


Figure 3. Diagram representing the GitHub environment

Creating a repository

The first thing to create a project (from now on, referred to as a repository) is to go to <https://github.com/> and create an account (if you do not have one already). Although in this lab we will be working with public repositories, note that you can apply to the Student Developer Pack to get unlimited private repositories with your university email (more info [here](#)).

To create a new repository, click on the plus “+” symbol on the upper right corner and click on “new repository”. Make it a public repository so that your classmates and the teachers can also see what you publish. Check the box “Initialise this repository with a README”. This will create a README.md file (you can change its name later, if you want), which will be the first file in your repository.

Before going any further, take a minute to register your repository in the repository form in Studium.

Now that you have created your *remote repository*, you need to create the *local repository* by “copying” the remote one to your computer. You can do this from the folder from which you will work during the labs, by using the **git clone** command (you will do this just the first time to set-up your local repository):

```
$ cd /home/<your_username>
$ git clone https://github.com/git_username/repository_name.git
```

If you clone the repository via https, like in the box above, you need to create a personal access token (PAT). To do so, click on your GitHub profile (upper right corner), then on “Settings” and from there on “Developer settings”. There you will get the option to create a PAT. When you create a PAT, give access to all repositories to it and make sure to grant it permissions at least to the code, commits, status checks and pull and push requests.

Important: Don't leave the page where you generated the PAT until you follow the next step (in UPPMAX):

```
$ git config --global credential.helper store
```

This will save your token once you input it.

Now copy the PAT. Try to pull or push from the local copy of your repository. When you are prompted to provide a password, paste the PAT. From now on, the PAT will be saved and you won't have to copy it again.

Now you are ready to work on your local repository!

An alternative is to generate a ssh key. In this case, you clone the repository using this code:

```
$ git clone git@github.com:<username>/<repository name>.git
```

You need to generate an ssh key (for example, a rsa key):

```
$ ssh-keygen -t rsa
```

Make sure to put the private key (the one that doesn't end with ".pub" in the `.ssh` directory (you can do `cd .ssh` from your home folder). Then, you can add the public key (`id_rsa.pub`) if you go to your GitHub profile > Settings > SSH and GPG keys.

Now you should be ready to work on your local repository! Note that you can generate other types of keys (it doesn't have to be an `rsa` key if you already have `id_rsa` in Uppmax), but it should be a default name (`id_<keytype>`), or otherwise Uppmax won't recognise your key.

Local vs remote repository

Your remote and local repositories should always be connected, and you can have as many copies of the local repository (e.g. in UPPMAX, your laptop, etc.) as you want. However, to avoid problems with the different versions, you will have to **git pull** everytime that you work from a different computer to synchronise your last working session. In this way, the changes tracked on your remote repository will be transferred to the current local repository:

```
$ git pull origin main
```

The name *origin* is a local nickname for your remote repository. It is the standard nickname so we suggest you use the same one. *Main* is the default main branch of your repository. We won't go into details here but feel free to read more about it.

In the same way, when you finish your working session you want to sync your local repository to the remote one. This time, you use the **git push** command:

```
$ git push origin main
```

Pull and push guarantee that your remote repository is always up-to-date even if you work from different computers.

Building up your repository

You can now start getting familiar with your working environment and create the different folders that you will use (i.e. `/data`, `/documents`, `/analyses`, etc.) based on your project plan and organisation. For instance, it could be useful to have a starting file where you explain the structure that your repository will have.

```
$ touch folders_structure.txt
```

Every time that you create/modify something in your working environment, git will notice it. You can keep track of these changes by using the **git status** command:

```
$ git status
```

```
# On branch main
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   folders_structure.txt
#
no changes added to commit (use "git add" and/or "git commit -a")
```

However, as you see from Git's answer after checking the status, although git knows that something has changed, it won't update those changes until you command it.

Updating your repository

Although git kind of knows what's going on, you need to actively transfer the data to the remote repository to keep track of every change. To do so, you need three steps.

First, you use the command **git add** to transfer your data to the staging area (the "limbo" area between your working copy and the local repository.) You have to use this option even when you modify an existing file (not only when you create it).

```
$ git add folders_structure.txt
```

The second step sends the data and changes from the staging area to your local repository, and you do it with the **git commit** command. A commit is a comment that describes what changes you have made since the last update (the last commit). This is an essential function to structure the state of your project and to be able to go back to any point. Therefore, it is strongly recommended that the text you add to the commit is clear and describes the step that you are updating.

```
$ git commit -m "Creating file containing the folder structure of my project"
```

Finally, you need to send the data to the remote repository. You can do this at the end of each session with the **git push** command that we have already introduced. It will send everything contained in the local repository since your last push.

Papers - short summary

In this section you will find brief information about the articles on which your projects will be based.

[Paper I](#)

RNA-seq and Tn-seq reveal fitness determinants of vancomycin-resistant *Enterococcus faecium* during growth in human serum

Xinglin Zhang, Vincent de Maat , Ana M. Guzmán Prieto , Tomasz K. Prajsnar , Jumamurat R. Bayjanov, Mark de Been , Malbert R. C. Rogers , Marc J. M. Bonten , Stéphane Mesnage , Rob J. L. Willems and Willem van Schaik

Enterococcus faecium is a commensal bacterium in the human gut that is associated with opportunistic bloodstream infections in immunocompromised hospitalized patients. Moreover, it has recently acquired resistance to multiple antibiotics, which represents a big public health concern. However, the growth and survival mechanisms of this opportunistic pathogen in the bloodstream have not been characterized. In this study you will identify what genes allow *E. faecium* to grow in human blood by different profiling techniques based on RNA-Seq and Tn-Seq.

Illumina PacBio Nanopore Genome Assembly RNA-Seq Tn-Seq Differential Expression

The main analyses included in this study are:

- Genome assembly on *E. faecium*
- Differential gene expression of *E. faecium* on human serum against rich medium using RNA-Seq data.
- Identification of genes that contribute to survival and growth in human serum using Tn-Seq data.

[Paper II](#)

Chromosome-level genome assembly of *Niphotrichum japonicum* provides new insights into heat stress responses in mosses

Xuping Zhou, Tao Peng, Yuying Zeng, Yuqing Cai, Qin Zuo, Li Zhang, Shanshan Dong, and Yang Liu

Bryophytes are a major and diverse lineage of land plants that can thrive in extreme environmental temperatures. The moss *Niphotrichum japonicum* (Grimmiaceae, Grimmiiales) can survive for extended periods under heat and drought conditions, making it a great system for studying the genetic basis of heat resilience. The authors of this paper assembled a chromosome-level genome of *Niphotrichum japonicum* using long and short reads

(**Nanopore and Illumina**) and data on genome-wide chromatin interactions (**Hi-C**). They also sequenced the moss transcriptome (**Illumina**) under heat stress conditions for various spans of time. The study uses comparative genomics to identify species-specific genes in *N. japonicum*, transcriptomics to find differentially expressed genes under heat stress and discusses heat tolerance mechanisms and evolution for the study system.

Illumina **Nanopore** **Genome Assembly** **Scaffolding with Hi-C** **RNA-Seq** **Differential Expression**

The main analyses included in this study are:

- Assembly and annotation of a part of the *N. japonicum* genome.
- Differential gene expression of *N. japonicum* against various time spans of heat stress using RNA-Seq data.

Evaluation

The grade for this lab will be sent individually to each student together with general feedback. In this course, the goal is to learn bioinformatics methods and how to apply them to obtain biological information.

To pass this part of the course (**grade 3**) you will have to fulfil the following points:

- **A set of basic methods** (see for each project below), and document them in your GitHub wiki. There will be certain deadlines along the course (see below), to make sure that everyone works at a good pace and will fulfil the minimum methods requirements.
- **GitHub wiki:** this will serve as your 'report'. Your text should be understandable to others (not only to yourself!) and contain the main steps needed to reproduce your work. In the wiki, you need to include:
 - the **project plan** (including specifics about data management),
 - the **methods** that you have been using for your project, and
 - the **biological interpretations** and conclusions of your results.

See "What should you include in your GitHub repository?" for details.

- **GitHub code:** you need to save all commands and scripts that you run to GitHub. It is important to keep this part updated because you might need to check previous steps in order to solve problems that you can have during the analyses. It also allows us to help you troubleshoot issues in your analyses.
- **10 min presentation + 5 discussion.** A good presentation should have a balanced structure covering aims and conclusions on your project. Your slides should be uploaded to Studium before the presentations. To ensure that you get proper feedback, make sure you time your presentation so that it doesn't exceed the given time. After 10 minutes, presentations can be stopped even if the presenter hasn't finished.

On Friday 22nd May, we will carefully check the state of your GitHub Wiki and check if you are ready to present. By Monday 25th May you will know if you are ready to present and you will upload your slides at 12:00 the latest. The GitHub Wiki and the presentation will be the basis (grade 3) of your final lab evaluation.

If you aim to get a higher grade (4-5), you need to fulfil additional criteria. You can get a 4 if:

- You have correctly answered the **questions in Studium** for a grade 4 that are related to your paper (see "Lab questions for a higher grade" in Studium).
- We deem that your **GitHub wiki and final presentation** are well-structured and display a clear understanding of the methods used and the meanings of the results.

You can get a 5 if you fulfill all the criteria above, and:

- You have correctly answered the **questions in Studium** for a grade 5 that are related to your paper (see “Lab questions for a higher grade” in Studium).
- You have performed **multiple extra analyses**.

Of course, these will also need to be uploaded to your GitHub account by the end of the lab session on Friday 22nd May in order to be included in the final evaluation.

Paper I - Zhang *et al.* 2017

Basic analyses:

- Genome assembly with PacBio reads.
- Assembly evaluation.
- Structural and functional annotation.
- Synteny comparison with a closely related genome.
- Reads preprocessing: trimming + quality check (before and after)
- RNA-Seq reads alignment against the assembled genome.
- Differential expression analysis between rich medium and heat-inactivated serum conditions.

Extra analyses:

- Genome assembly with Illumina and Nanopore reads.
- Assembly evaluation (extra methods).
- Annotation refinement
- Plasmid identification.
- SNP calling.
- Evaluate antibiotic resistance potential.
- Identify essential genes for growth in human serum based on the Tn-Seq data analysis.

Paper II - Zhou *et al.* 2023

Basic analyses:

- Reads preprocessing: trimming + quality check (before and after)
- Genome assembly of Nanopore reads.
- Polish the assembly with Illumina reads.
- Assembly quality assessment with Busco and Quast.
- Structural and functional annotation.

- Differential expression analysis (including different comparisons)

Extra analyses:

- Assembly with different parameters or assemblers.
- Assembly evaluation with Merqury.
- Scaffold assembly with HiC data.
- Assemble and annotate the chloroplast genome
- Find unique genes in *N. japonicum* and DEGs in the Chloroplast genome
- Gene enrichment analysis for unique and/or DEGs in *N. japonicum*.
- Plot synteny with a related genome, either based on DNA sequence identity or on protein position
- Identify duplicated genes

Practical Information About Software

The choice of software to use will depend on the kind of data you have (type of organism, DNA or RNA, type of reads...), and it's something very important to consider beforehand. Here you have a list of software that we have tested for all the suggested papers. We have also included tips for certain software that you will need to keep in mind when you run your own analyses. Of course you are free to use other tools to perform your analyses, but we cannot guarantee that they will work nicely for your data. It is still possible that you get errors when running the analyses with the recommended software. If that happens, keep calm, **read what the error says** and google it if you don't know what to do. Probably it has happened to other people and the solution might be out there. It is a good learning method for you to experience how real research feels and try to figure out your own way.

This section also contains some questions about each analysis. It is very important that you take some time to think about them since it will give you a better understanding of your own data, what you are doing and will also help you to interpret your results. Even though not all the questions might apply to the specific analyses you are doing, it is a good exercise to think about them.

Reads quality control

It is not recommended to use the raw data directly in your analyses without checking their quality, because if the quality is not optimal, you will get errors or artefacts in the subsequent analyses. So the very first thing you should always do when working with reads is to assess the quality of your data. If the quality of your data is not good enough you might need to remove those parts of the reads with low quality or adapters still present on your reads. After that step you should always check the quality of your data again to make sure that all the issues are solved.

- **FastQC** aims to provide a simple way to check the quality of short reads coming from high throughput sequencing pipelines. It performs a modular set of analyses to generate a report that you can use to have a quick impression of whether your data have any problems of which you should be aware before doing any further analysis.

Reads preprocessing

An essential step when working with short reads is removing adapters and low quality base-calls from the reads. Sometimes this is done directly by the assembler, but other times this is a manual step that cannot be skipped. Since this analysis can be time-consuming and occupy much of the allocated storage space, we are providing you with already trimmed reads for some projects. However, you might need to do it anyway for specific samples.

- **Trimmomatic** is a fast, multithreaded command line tool that can be used to trim and crop Illumina (FASTQ) data as well as to remove adapters. These adapters can pose a real problem depending on the library preparation and downstream application.

Notes:

- *Trimmomatic will perform the different filtering steps in the same order as specified on the command line. In this case the order of the settings is important!*
- *Now you can run the command directly, without specifying the path to the .jar file: trimmomatic PE. Alternatively, you can use: java -jar \$TRIMMOMATIC_ROOT/trimmomatic.jar PE*

Genome and Metagenome Assembly

There are different assemblers that can be used depending on the characteristics of your sequencing reads (short or long reads) and the kind of data (DNA, RNA, metagenome...). Here is a list of the main assembly software that can be useful for you:

- **DNA assembly**
 - **Flye** is a *de novo* assembler for long reads (PacBio and Nanopore). It can be used on any genome, from small bacterial genomes to large eukaryotic genome assemblies.
 - **Hifiasm** is a *de novo* assembler for long reads (PacBio and Nanopore) originally designed for PacBio, which has generally higher quality scores than Nanopore. It has largely been tested on Eukaryotic genomes.
 - **Canu** is a *de novo* whole-genome shotgun (WGS) assembler especially designed for long-read sequencing technologies.

Notes:

- *If you use a version of Canu **below 2.0**, the software may detect that the quality of the reads is too low and stop the assembly. To avoid that, you can use the option 'stopOnReadQuality=false'.*
- *In Canu **v2.0 or later**, set 'useGrid=false' or provide 'gridOptions'.*
- *Canu needs 4 cores to run, set 'maxThreads=4'*
- **SOAPdenovo** is a short-read assembly method that can produce large genomes (human-size).

- **Spades** is an assembly toolkit capable of providing hybrid assemblies (combining short and long reads, i.e. Illumina + PacBio). It can work with paired-end reads, mate-pairs and unpaired reads.

Notes:

- *Spades requires A LOT of memory and LONG running times. In order to reduce the running time set just one kmer size (by default Spades will use several) and never use the option --careful in this lab. You can also skip the error correction part, but in those cases Spades will use even more memory.*
- **Pilon** is a software that can improve draft assemblies and find variation among strains. It uses short reads alignment against long reads assemblies to identify inconsistencies between the input genome and the mapped reads.
- **RepeatMasker** is a program that screens DNA sequences and detects repeats and low complexity regions. In order to predict genes accurately in a novel genome, the genome should be masked for repeats. There are two types of masking on DNA sequences, softmasking (i.e. putting repeat regions into lower case letters and all other regions into upper case letters) and hardmasking (i.e. replacing letters in repetitive regions by the letter N for unknown nucleotide)
- **RNA assembly**
 - **Trinity** is an assembler useful for the *de novo* reconstruction of transcriptomes from RNA-seq data.

Note:

- *On Uppmax use the version 2.4.0.*

Assembly evaluation

Any assembly of our reads will most likely not be optimal. There are different strategies to test how good an assembly is, and it depends on you (or the available references to compare with) to decide which one to use.

- **BUSCO** (Benchmarking Universal Single-Copy Orthologs) is a widely used tool for assessing genome assembly completeness based on evolutionarily conserved genes. A protein .fasta file is usually given to BUSCO.
- **QUAST** evaluates how good a newly generated genome assembly is. It can work both with and without a reference genome to compare with, and accepts multiple assemblies.

- **MUMmerplot** uses the output from mummer, nucmer, promer (whole-genome alignment), and generates an alignment dot-plot comparing two aligned assemblies.

Note:

- *If the data contain several contigs, use the parameters -R and -Q. They make the plot interpretation easier by delimiting the different contigs.*
- *If your assembly contains too many contigs, remove the smallest ones or concatenate your assembly to get a readable plot.*

Annotation

Once we have assembled our genome/metagenome, the next step is to predict what genetic elements it encodes. A good complement to our annotation is attaching biological information to our predicted genetic elements, such as the biochemical function they perform, or the biological process they are involved in. The best strategy is to use software pipelines that combine several programs in an automated way. Although they are a great starting point, these automatic approaches usually make mistakes and might require extensive manual curation.

- **Prokka** is a software pipeline that combines different tools for the annotation of prokaryotic genomes. It combines both structural and functional annotation by predicting and identifying genetic elements encoded in the genome.
 - *Be aware if you are annotating Archaea, since the default for Prokka is bacterial annotation. Check the “kingdom” option.*
- **BRAKER3** is a structural annotation pipeline that combines GeneMark-ET and AUGUSTUS, that uses genomic and RNA-Seq data to automatically generate full gene structure annotations in novel genomes.

BRAKER3 is installed as a docker container that has been developed to run with singularity.

Some arrangement needs be done at the working directory on UPPMAX:

- *AUGUSTUS needs writing access to the configuration file:*

```
chmod a+w -R /home/<your_username>/<working_directory>/augustus_config/species/
```

- GeneMark key needs to be copied to the home directory:

```
cp -vf /sw/bioinfo/GeneMark/4.33-es/snowy/gm_key $HOME/gm_key
```

Some tips to successful gene prediction:

- Copy configuration file for AUGUSTUS

```
source $AUGUSTUS_CONFIG_COPY
```

- BRAKER needs the following flags of the paths to the softwares

```
-- AUGUSTUS_CONFIG_PATH=/home/<your_username>/augustus_config
-- AUGUSTUS_BIN_PATH=/sw/bioinfo/augustus/3.4.0/snowy/bin
-- AUGUSTUS_SCRIPTS_PATH=/sw/bioinfo/augustus/3.4.0/snowy/scripts
-- GENEMARK_PATH=/sw/bioinfo/GeneMark/4.33-es/snowy
```

- Species needs to be updated with `--useexisting` flag once the new species file is created.
- If the genome is masked, use the `--softmasking` flag of `braker.pl`
- BRAKER suggested to be run with 8 cores
- **EggNOGmapper** is a tool for fast functional annotation of already predicted sequences (works both for genes and proteins) using precomputed eggNOG-based orthology entries. It has an online web server.

Note:

- There is a bug in the online version and does not retrieve the best hit when searching with Diamond. Choose HMM instead of Diamond to get this information.

Homology search

Sequence homology shows us how similar two or more organisms are by comparing them at the gene or protein level.

- **Blastn** is a local alignment algorithm included in BLAST that allows you to compare a nucleotide query against a database, or two nucleotide sequences against each other. It can be used from a single gene to the entire genome.

Mapping

Aligning reads against a reference can be something you want to do either to determine the reads coverage, to improve or correct an assembly, or to account for differential expression of certain genes, among others. Something important you need to consider here is what kind of organism you are working with, because that will determine what software is more suitable for you.

- **BWA** is a package for mapping low-divergent sequences against a reference genome, as long as they do not contain an exon-intron structure. It will be extensively used across these labs to map reads against assembled genomes with different purposes.
- **STAR** Spliced Transcripts Alignment to a Reference (STAR) is a fast RNA-seq read mapper. It detects splice-junctions between exons and fusion reads.

Post-mapping analyses

This section includes tools that you might need to use to manipulate mapping files and/or extract specific information from them.

- **SAMtools** provides various utilities for manipulating alignments in the SAM format, including sorting, merging, indexing and generating alignments in a per-position format.

Notes:

- *Alignment files in SAM format are extremely heavy, so generating BAM files is strongly recommended.*
- *In the process of obtaining a position-sorted final alignment, it is easy to generate several intermediate and dispensable files of large sizes. To avoid this, make sure that you pipe all your SAMtools inputs and outputs in one command to obtain directly a sorted BAM output file when you align your reads against an assembly.*
- **BCFtools** is a set of utilities very useful to find SNPs and indels in our assembly by analysing short reads mapping against a reference assembly.

Read counting

After mapping RNA reads to a reference, you might want to know how many reads map to different genomic features (i. e. genes) or to specific contigs, for example. That's an essential step for looking at the expression of particular genes and doing a differential expression analysis.

- **HTSeq** is a Python package that will be used for counting reads that map on genomic features. It is slower than the alternative, but its output data are easier to load into DESeq2.

Notes:

- *BAM alignment files must be previously sorted by position*
- *The features file (.gff) must not contain nucleotide sequences. You might need to manually remove them to use this tool.*
- **featureCounts** is another program that counts reads mapping to genomic features. It is faster than HTSeq, but more difficult to load into DESeq2.

Notes:

- *BAM alignments must also be sorted, and feature files must not have any nucleotide sequences.*
- *featureCounts is part of the Subread package. In UPPMAX, there is no package called featureCounts; you must load the module for Subread instead.*

Expression analyses

To compare gene expression between samples, first, we need to count the amount of reads that map against each gene, and then we determine the differences between them. Once you have the count table with the number of reads mapped to each gene (see “Read counting” section), you can run the differential expression analysis on it:

- **DESeq2** is an R package included in Bioconductor to estimate variance-mean dependence in the counted reads against gene features under different conditions, and test for differential expression based.

Note:

- *If you're having problems with version incompatibility, to install DESeq2 for R 3.5 use:*

If (!requireNamespace("BiocManager", quietly = TRUE))

```
install.packages("BiocManager")
```

```
BiocManager::install("DESeq2", version = "3.8")
```

OBS!: Installation can take a while. Do it in advance, while you wait for other analyses to run.

Visualisation of the genomes

Finally, sometimes you will need to visually analyse and inspect genome features (annotation, genome comparisons, mapping reads, etc). Here are a couple of tools that will be useful for that purpose.

- **IGV** is a visualisation tool for interactive exploration of large, integrated genomic datasets useful to analyse different data types (short and long reads and genomic annotation).

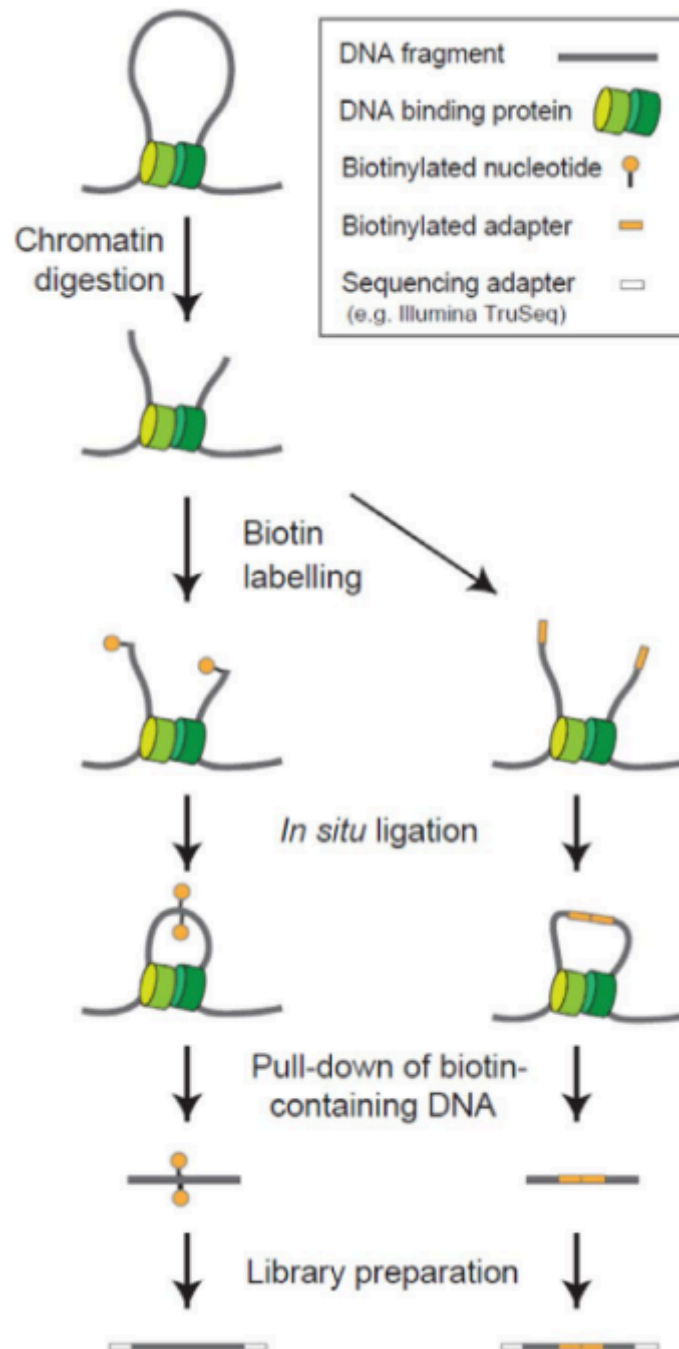
Notes:

- *If you want to work with IGV locally (recommended) you need to download it from their [website](#) (you need to register first).*
 - *When you look at the different tracks, keep an eye on the scale since this could be misleading. The scale is shown in the top-left corner of each track between square brackets (e.g [0-2394]). You can change the scale of a track to improve the visualisation of it by right clicking on the track and ticking on "Autoscale".*
 - *You can also select several tracks (for example all mRNA data) and set the same scale for all of them, by right clicking on the selected tracks and "**Group Autoscale**". In this way you could have different scales for different types of data (e.g. one for the mRNA data and one for the ChIP-seq data).*
- **Artemis Comparison Tool (ACT)** is another visualisation tool especially designed for displaying pairwise comparisons between two or more DNA sequences (obtained with BLAST).

Genome scaffolding

There exist genome sequencing technologies that allow us to study chromatin structure. Studying chromatin structure can allow us to identify contigs in our assembly that are in close physical proximity to each other, allowing us to place contigs of our assembly into pseudomolecules that are representative of the chromosomes of our organism of interest. Chromatin structure can also provide insights into chromatin organization and gene regulation. For example we can tell apart euchromatin from heterochromatin, or we can find which enhancers activate the expression of upregulated genes. One type of sequencing that

we use to study chromatin structure is Hi-C. For Hi-C sequencing, we cross-link chromatin regions that are in proximity to each other using formaldehyde, we shear the DNA and we use Illumina paired-end sequencing to get the sequence of the ligated fragments. What happens is that fragments from the same chromosome will end up ligating more frequently than with fragments from other chromosomes. Also, fragments of the same chromosome will ligate more frequently with fragments that have a smaller genomic distance, than with fragments with greater genomic distance. By mapping the ligated fragments on our contigs we can reconstruct the structure of the chromosome from which the fragments came from. It is important that we filter out secondary alignments and that we mark PCR/optical duplicates, things that can introduce bias to our scaffolding result. This can be done using samtools/Picard or other tools of your choice. For scaffolding we recommend using Yaho which is easy and fast to run.



Overview of the workflow used for Hi- C library preparation. Figure from Yamaguchi et al. 2021.

For more information on scaffolding, you can read:

Lajoie BR, Dekker J, Kaplan N. 2015. The Hitchhiker's guide to Hi-C analysis: Practical guidelines. *Methods* 72:65–75.

Yamaguchi K, Kadota M, Nishimura O, Ohishi Y, Naito Y, Kuraku S. 2021. Technical considerations in Hi-C scaffolding and evaluation of chromosome-scale genome assemblies. *Mol Ecol* 30:5923–5934.

Appendix I: Software list

In addition to looking at the manuals, do `module help <name of module>/<version>` to check for the usage of the software in Uppmax. This command will provide a short description of how to run the software, including the path to the executable files.

Analysis	Type	Software	Installed	Tutorial
Reads Quality Control	Illumina	FastQC	UPPMAX	https://www.bioinformatics.babraham.ac.uk/projects/fastqc/
Reads preprocessing	Illumina	Trimmomatic	UPPMAX	http://www.usadellab.org/cms/uploads/supplementary/Trimmomatic/TrimmomaticManual_V0.32.pdf
DNA assembly	Illumina	Spades	UPPMAX	https://github.com/ablab/spades
	Illumina	GetOrganelle	UPPMAX	https://github.com/Kinggerm/GetOrganelle
	PacBio or Nanopore	Flye	UPPMAX	https://github.com/fenderglass/Flye/blob/flye/docs/USAGE.md
	PacBio or Nanopore	Hifiasm	UPPMAX	https://github.com/chhylp123/hifiasm?tab=readme-ov-file#ononly
	Assembly Improvement	Pilon	UPPMAX	https://github.com/broadinstitute/pilon/wiki
	Assembly Improvement	DeepVariant	UPPMAX	https://github.com/google/deepvariant
	Assembly Masking	RepeatMasker	Online UPPMAX	https://www.repeatmasker.org/cgi-bin/WEBRepeatMasker
	PacBio or Nanopore	canu	UPPMAX	https://canu.readthedocs.io/en/latest/tutorial.html
RNA assembly	Illumina RNA	Trinity	UPPMAX	https://github.com/trinityrnaseq/trinityrnaseq/wiki/Running-Trinity
Assembly evaluation	Fasta	Quast	Locally, UPPMAX	http://quast.sourceforge.net/docs/manual.html
		BUSCO	UPPMAX	https://busco.ezlab.org/busco_userguide.html
		Merquy	UPPMAX	https://github.com/thegenemyers/MERQURY.FK
		MUMmerplot	UPPMAX	http://mummer.sourceforge.net/manual/#mummer

				plot
		BCFtools	UPPMAX	https://samtools.github.io/bcftools/bcftools.html
Annotation	Bacteria	Prokka	Locally, UPPMAX	https://github.com/tseemann/prokka#invoking-prokka
	Eukaryotes	BRAKER	UPPMAX (singularity)	https://tseemann/prokka:;zap:Rapid_prokaryotic_genome_annotation_GitHub//github.com/Gaius-Augustus/BRAKER#running-braker
	All	eggNOGmapper	Online UPPMAX	https://github.com/eggno-gdb/eggno-gdb-mapper/wiki/
Mapping	DNA & bacterial RNA	BWA	UPPMAX	http://bio-bwa.sourceforge.net/bwa.shtml
	Eukaryotic RNA	hisat2	UPPMAX	http://daehwankimlab.github.io/hisat2/manual/
	Eukaryotic RNA	STAR	UPPMAX	https://github.com/alexdobin/STAR https://hbctraining.github.io/Intro-to-rnaseq-hpc-O2/lessons/03_alignment.html
Differential Expression	Count features	Htseq, featureCounts	UPPMAX	https://htseq.readthedocs.io/en/release_0.9.1/count.html#count
	Comparison	DESeq2 (R library)	Locally, UPPMAX	https://rdr.io/bioc/DESeq2/man/
Gene enrichment analysis		rrvgo	Locally	https://bioconductor.org/packages/devel/bioc/vignettes/rrvgo/inst/doc/rrvgo.html
Protein clustering	Find Homologs, and unique genes. Comparative genomics	Orthofinder	UPPMAX	https://github.com/davidepms/OrthoFinder https://github.com/davidepms/OrthoFinder/blob/master/OrthoFinder-manual.pdf
Visualisation	Genomes: (Reads & Annotation)	IGV	Locally, UPPMAX	http://software.broadinstitute.org/software/igv/userguide
		Artemis	Locally, UPPMAX	https://www.sanger.ac.uk/tool/artemis/

		Tablet	Locally, UPPMAX	https://ics.hutton.ac.uk/tablet/
	Genomes: BLAST comparisons	ACT	Locally, UPPMAX	https://www.sanger.ac.uk/tool/artemis-comparison-tool-act/
	Whole genome alignment	D-Genies	Online, locally	https://dgenies.toulouse.inra.fr/ https://github.com/genotoul-bioinfo/dgenies
Homology search	Nucleotide sequences	Blastn (BLAST)	Locally, UPPMAX	https://www.ncbi.nlm.nih.gov/books/NBK279690/

Appendix II: Estimated running time for analyses

Analysis	Software	Running time
Reads Quality Control	FastQC	Paper I: ~ 10 min
Short reads preprocessing	Trimmomatic	Paper I: ~ 50 min per file (1 core)
DNA assembly	Spades	Paper I: (short reads + long reads) ~ 2 h (1 core)
	Flye	Paper II: 5h (16 cores)
	Canu	Paper I: ~ 2,5 h (4 cores) Paper II: ~ 17 h (4 cores)
	Pilon	Paper II: ~ 12h (2 cores, including read mapping)
	Deepvariant	Paper II: ~ 13h (4 cores, including read mapping)
	RepeatMasker	Paper II: ~ 1h and 40 mins (4 cores)
	GetOrganelle	Paper II: ~ 2h (4 cores) for cpDNA
Assembly Scaffolding	Yahs	Paper II: ~ 1h (4 cores)
RNA assembly	Trinity	Paper II: ~ 5,5 h (4 cores)
Assembly evaluation	Quast	Paper I: < 15 min (1 core) Paper II: < 15 min (1 cores)
	BUSCO	Paper I (extra) Paper II
	MUMmerplot	Paper I: < 5 min (1 core)
	BCFtools	Paper I: ~ 90 min (1 core)
Annotation	Prokka	Paper I: < 5 min (1 core)
	BRAKER2	Paper II: ~ 20 mins to map RNA, ~3h to run braker2 (16 cores)
	eggNOGmapper	Paper I (extra): ~ 13 h (2 cores) Paper II: ~ 2h (1 core)
Aligner	BWA	Paper I (paired-end reads): ~ 30 min (1 core) Paper I (single reads): < 15 min (1 core) Paper II: ~ 1 h (2 cores)
	hisat2	Paper II: ~ 20 mins (16 cores)
	STAR	Paper II: 15 min (8 cores)
Differential Expression	Htseq	Paper I (paired-end reads) ~ 2-7 h (1 core) Paper I (single reads) < 10 min (1 core)
	featureCounts	Paper II: ~ 12h to map 6 RNA samples and to count (1 core)

	Deseq2 (R library)	Variable (you write your own R script), usually a few minutes to run
Gene enrichment analysis	rrvgo	Variable (you write your own R script), usually a few minutes to run
Protein clustering	Orthofinder	Paper II: ~ 15 mins -1h (4 cores) for proteomes from two species
Genome comparison	BLAST	Paper I: < 1 min (2 cores)

Appendix III: Analyses checkpoints

To ensure that everyone will get to the end of the labs with the minimum analyses expected, you have to plan accordingly to meet the following deadlines, note that there are also three compulsory lab sessions:

Day	Hours	Eukaryotes	Prokaryotes	Reservation code
27/3	2	Compulsory Seminar	Compulsory Seminar	
08/4	4	Compulsory GitHub/Biology	Compulsory GitHub/Biology	
10/4	4	Project plan	Project plan	uppmx2026-1-61_1
15/4	4	Genome assembly	Genome assembly	uppmx2026-1-61_2
16/4	4	Compulsory Genome Assembly	Compulsory Genome assembly	uppmx2026-1-61_3
21/4	4	Genome Polishing	Assembly evaluation	uppmx2026-1-61_4
24/04	4	Genome Masking		uppmx2026-1-61_5
28/4	4	Compulsory Annotation	Compulsory Annotation	uppmx2026-1-61_6
5/5	4	Annotation	Comparative genomics	uppmx2026-1-61_7
8/5	4		RNA trimming	uppmx2026-1-61_8
11/5	4	Compulsory RNA mapping	Compulsory RNA mapping	uppmx2026-1-61_9
13/5	4	Read counting	Read counting	uppmx2026-1-61_10
19/5	4	Compulsory DE analysis	Compulsory DE analysis	uppmx2026-1-61_11
22/5	4	Wiki	Wiki	uppmx2026-1-61_12
26/5	7	<i>Project presentations</i>		